

Contrato API

Servicio de Despacho y Logística

Grupo 6

Especificación de Endpoints REST

Contratos JSON de Eventos

Matriz de Dependencias

Proyecto: Marketplace en la Nube
Versión API: v1.1
Base URL: <https://g6-despacho.onrender.com/api/v1>
Fecha: 17 de Junio de 2026

Índice

I	Contrato REST API	2
1.	Responsabilidad del Servicio y Dominio	2
1.1.	Objetivo	2
1.2.	Qué resuelve este servicio	2
1.3.	Qué queda fuera de alcance	2
1.4.	Dueño del Dato (Ownership)	2
2.	Información General	2
2.1.	Convenciones	2
2.2.	Resumen de Endpoints	3
3.	Modelo de Datos: Shipment	3
3.1.	Estados válidos	3
4.	Endpoints Detallados	3
4.1.	Crear Despacho	4
4.2.	Cotizar Envíos	5
4.3.	Consultar Despacho por ID	6
4.4.	Buscar Despacho por Order ID	6
4.5.	Actualizar Estado del Despacho	7
4.6.	Listar Despachos	7
4.7.	Health Check	8
5.	Estructura Estándar de Errores	8
II	Contrato de Eventos JSON	10
6.	Mecanismo de Comunicación (Pub/Sub)	10
6.1.	Flujo de Eventos	10
7.	Formato Estándar del Evento	10
8.	Catálogo de Eventos	10
8.1.	ShipmentCreated	10
8.2.	ShipmentInTransit	11
8.3.	ShipmentDelivered	12
8.4.	ShipmentCancelled	12
8.5.	ShipmentFailed	13
8.6.	ShipmentReturned	13
III	Contratos Inter-Grupo	15
9.	Contrato con G5 (Pedidos o Order Management) — Entrada	15
9.1.	Flujo de Integración	15
9.2.	JSON que G5 nos envía	15
9.3.	JSON que G6 le responde a G5	15
9.4.	Preguntas pendientes para G5	15

10. Contrato con G8 (Pago Simulado y Notificaciones) — Salida	15
10.1. Flujo de Integración	16
10.2. JSON que G6 publica en el Event Broker (para G8)	16
10.3. Comportamiento esperado de G8 (Pago Simulado y Notificaciones)	16
10.4. Preguntas pendientes para G8	17
11. Contrato con G7 (Reportería, Batch y Streaming) — Salida	17
11.1. Opciones de Integración	17
11.2. Datos relevantes para G7	17
12. Contrato con G1 (Frontend) — Consulta	17
 IV Matriz de Dependencias y Arquitectura	 18
13. Matriz de Dependencias	18
14. Diagrama de Arquitectura (C4 — Nivel 2: Contenedores)	18
15. Esquema SQL de la Base de Datos	19
16. Supuestos, Riesgos y Mitigaciones	20
16.1. Supuestos	20
16.2. Riesgos y Mitigaciones	20
16.3. Manejo de Errores	21
17. Historial de Versiones	21

Parte I

Contrato REST API

1. Responsabilidad del Servicio y Dominio

1.1 Objetivo

El servicio de **Despacho y Logística (G6)** es responsable de gestionar el ciclo de vida completo de los envíos dentro del ecosistema del Marketplace en la Nube.

1.2 Qué resuelve este servicio

- Crear despachos a partir de pedidos confirmados por G5.
- Asignar un identificador único (**shipment_id**) a cada envío.
- Gestionar la máquina de estados del envío (**PENDING** → **IN_TRANSIT** → **DELIVERED**, etc.).
- Publicar eventos de cambio de estado para que G8 notifique al usuario y G7 genere reportes.
- Exponer endpoints REST para consulta de tracking (G1) y listados (G7).
- Almacenar y administrar la tabla **shipments** como fuente de verdad.

1.3 Qué queda fuera de alcance

- **Gestión de pedidos:** responsabilidad de G5 (Order Management).
- **Procesamiento de pagos:** responsabilidad de G8 (Pago Simulado).
- **Generación de reportes:** responsabilidad de G7 (Reportería y Batch).
- **Interfaz de usuario:** responsabilidad de G1 (Frontend).
- **Autenticación y autorización:** delegada a G1/G8.

1.4 Dueño del Dato (Ownership)

Dato	Dueño	Acceso G6	Observación
shipments	G6 (propio)	Lectura/Escritura	Fuente de verdad del envío.
order_id	G5 (ajeno)	Solo lectura	Recibido en el POST, nunca modificado.
customer_name	G5 (ajeno)	Solo lectura	Copiado del pedido para el envío.
payment_status	G8 (ajeno)	No accede	G6 no consulta ni modifica pagos.
report_data	G7 (ajeno)	No accede	G7 consume datos vía REST/eventos.

Cuadro 1: Ownership de datos: qué datos son propios y cuáles son consultados.

2. Información General

2.1 Convenciones

- Todos los endpoints requieren los siguientes headers obligatorios:
 - X-Request-Id: uuid
 - X-Correlation-Id: uuid
 - X-Consumer: group-name

Campo	Valor
Título	API de Despacho y Logística — Grupo 6
Versión	1.0.0
Base URL	https://g6-despacho.onrender.com/api/v1
Formato	JSON (application/json)
Autenticación	Ninguna (proyecto académico)
Encoding	UTF-8

- Idempotency-Key: uuid
- Todos los endpoints retornan `application/json`.
- Las fechas usan formato ISO 8601: `YYYY-MM-DDTHH:mm:ssZ`.
- Los IDs de shipment usan formato `SHP-<uuid>` (generados por nuestro servicio).
- Los errores siguen estructura estándar (ver Sección 5).

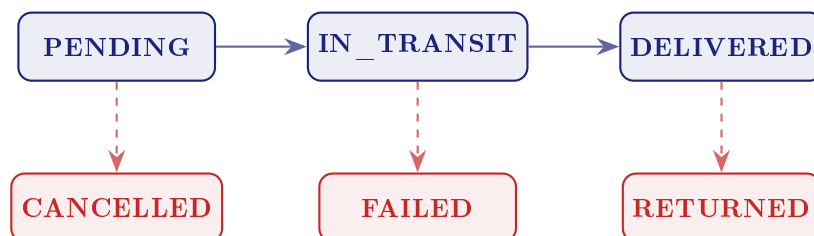
2.2 Resumen de Endpoints

Método	Ruta	Descripción	Consumidor
POST	/shipments	Crear un nuevo despacho	G5
GET	/shipments/{id}	Consultar estado de un despacho	G1, G5
GET	/shipments?order_id={x}	Buscar despacho por pedido	G5
PATCH	/shipments/{id}	Actualizar estado del despacho	Interno
GET	/shipments	Listar todos los despachos	G7
GET	/health	Health check del servicio	Todos

Cuadro 2: Resumen de endpoints expuestos por el Grupo 6.

3. Modelo de Datos: Shipment

3.1 Estados válidos



4. Endpoints Detallados

Campo	Tipo	Ejemplo	Req.	Descripción
shipment_id	string	SHP-a1b2c3d4	—	ID único generado por G6.
order_id	string	ORD-20260611-001	Sí	ID del pedido (de G5).
customer_name	string	María G.	Sí	Nombre del destinatario.
address	string	Av. Prov. 1234	Sí	Dirección de entrega.
city	string	Santiago	Sí	Ciudad de destino.
weight_kg	number	3.2	Sí	Peso del paquete en kg.
status	string	PENDING	—	Estado actual.
created_at	datetime	2026-06-11T14:00Z	—	Fecha de creación.
updated_at	datetime	2026-06-11T15:30Z	—	Última actualización.
estimated_delivery	datetime	2026-06-14T18:00Z	—	Fecha estimada.

Cuadro 3: Esquema del recurso Shipment. “Req.” indica si es requerido en el POST.

Estado	Valor	Tipo	Transiciones válidas
Pendiente	PENDING	Inicial	→ IN_TRANSIT, CANCELLED
En tránsito	IN_TRANSIT	Normal	→ DELIVERED, FAILED
Entregado	DELIVERED	Final	→ RETURNED
Cancelado	CANCELLED	Final	Ninguna (terminal).
Fallido	FAILED	Final	Ninguna (terminal).
Devuelto	RETURNED	Final	Ninguna (terminal).

Cuadro 4: Tabla de transiciones de estado.

4.1 Crear Despacho

POST /api/v1/shipments

Descripción: Crea una nueva orden de despacho. Llamado por **G5 (Pedidos o Order Management)** cuando un pedido ha sido confirmado y pagado.

Request Body (application/json):

```
{
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "packages": [
    { "origin_cd": "NORTE", "weight_kg": 2.5, "dimensions_cm": {
      "length": 40, "width": 30, "height": 20 } },
    { "origin_cd": "CENTRO", "weight_kg": 1.0, "dimensions_cm": {
      "length": 15, "width": 10, "height": 5 } }
  ]
}
```

Campos requeridos:

Campo	Tipo	Req.	Validaciones
order_id	string	Sí	No vacío. Único.
customer_name	string	Sí	No vacío. Máx. 200 chars.
address	string	Sí	No vacío. Máx. 500 chars.
city	string	Sí	No vacío. Máx. 100 chars.
weight_kg	number	Sí	Mayor a 0. Máx. 500 kg.

Response — 201 Created:

```
[
  \"SHP-1\",
  \"SHP-2\"
]
```

Errores:

Código	Nombre	Causa
400	Bad Request	JSON mal formado o campos faltantes.
409	Conflict	Ya existe despacho para ese order_id.
422	Unprocessable Entity	Datos inválidos (ej: peso negativo).

4.2 Cotizar Envíos

POST /api/v1/shipments/quotes

Descripción: Cotiza envíos según peso volumétrico y orígenes. Llamado por **G4 (Inventario y Checkout)**. Fórmula: $\text{Peso Volumétrico (kg)} = (\text{Largo_cm} * \text{Ancho_cm} * \text{Alto_cm}) / 4000$.

Request Body (application/json):

```
{
  "city": "Santiago",
  "address": "Av. Providencia 1234",
  "packages": [
    {
      "origin_cd": "NORTE",
      "weight_kg": 2.5,
      "dimensions_cm": { "length": 40, "width": 30, "height": 20 }
    }
  ]
}
```

Response — 200 OK:

```
{
  "total_shipping_cost": 6600,
}
```

```
"currency": "CLP"
}
```

4.3 Consultar Despacho por ID

GET /api/v1/shipments/{shipment_id}

Descripción: Retorna estado actual y datos completos de un despacho. Llamado por **G1** (Frontend) y **G5** (Pedidos o Order Management).

Response — 200 OK:

```
[
  {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "customer_name": "María González",
    "address": "Av. Providencia 1234, Depto 56",
    "city": "Santiago",
    "weight_kg": 3.2,
    "status": "IN_TRANSIT",
    "created_at": "2026-06-11T14:00:00Z",
    "updated_at": "2026-06-11T14:30:00Z",
    "estimated_delivery": "2026-06-14T18:00:00Z"
  }
]
```

Errores:

Código	Nombre	Causa
404	Not Found	No existe despacho con ese ID.

4.4 Buscar Despacho por Order ID

GET /api/v1/shipments?order_id={order_id}

Descripción: Busca un despacho por el ID de pedido de G5.

Response — 200 OK:

```
[
  {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "status": "PENDING",
    "created_at": "2026-06-11T14:00:00Z",
    "estimated_delivery": "2026-06-14T18:00:00Z"
  }
]
```


]

Errores:

Código	Nombre	Causa
400	Bad Request	Falta parámetro <code>order_id</code> .
404	Not Found	No existe despacho para ese pedido.

4.5 Actualizar Estado del Despacho

PATCH /api/v1/shipments/{shipment_id}

Descripción: Actualiza el estado de un despacho. Uso **interno** (simulación automática) o por **G5** para cancelar.

Request Body:

```
{
  "status": "IN_TRANSIT"
}
```

Response — 200 OK:

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "status": "IN_TRANSIT",
  "previous_status": "PENDING",
  "updated_at": "2026-06-11T14:30:00Z"
}
```

i Nota

Al cambiar el estado exitosamente, el servicio emite automáticamente el evento correspondiente hacia G8 (ver Parte II).

Errores:

Código	Nombre	Causa
400	Bad Request	Estado no es un valor válido.
404	Not Found	No existe despacho con ese ID.
409	Conflict	Transición no permitida (ej: <code>DELIVERED</code> → <code>PENDING</code>).

4.6 Listar Despachos

GET /api/v1/shipments

Descripción: Lista de despachos con filtros. Para **G7 (Reportería, Batch y Streaming)**.

Query Parameters (opcionales):

Param	Tipo	Default	Descripción
status	string	(todos)	Filtrar por estado.
limit	int	50	Máx. resultados.
offset	int	0	Paginación.

Response — 200 OK:

```
{
  "total": 142,
  "limit": 50,
  "offset": 0,
  "shipments": [
    {
      "shipment_id": "SHP-a1b2c3d4",
      "order_id": "ORD-20260611-001",
      "status": "DELIVERED",
      "city": "Santiago",
      "created_at": "2026-06-11T14:00:00Z",
      "updated_at": "2026-06-11T16:00:00Z"
    }
  ]
}
```

4.7 Health Check

GET /api/v1/health

```
{
  "status": "ok",
  "service": "g6-despacho",
  "version": "1.0.0",
  "timestamp": "2026-06-11T14:00:00Z"
}
```

5. Estructura Estándar de Errores

Todos los errores siguen el mismo formato base. Opcionalmente, se puede incluir un campo "details" para proveer más contexto de validación:

```
{
  "timestamp": "2026-06-11T14:05:00Z",
  "status": 422,
```

```
{
  "code": "VALIDATION_ERROR",
  "message": "El campo weight_kg debe ser mayor a 0.",
  "correlationId": "req-123e4567"
}
```

status	code	Uso
400	BAD_REQUEST	JSON malformado, campos faltantes.
404	NOT_FOUND	Recurso no encontrado.
409	CONFLICT	Duplicado o transición inválida.
422	VALIDATION_ERROR	Datos que no pasan validación.
500	INTERNAL_ERROR	Error inesperado del servidor.

Cuadro 5: Códigos de error estandarizados.

Parte II

Contrato de Eventos JSON

6. Mecanismo de Comunicación (Pub/Sub)

La arquitectura de eventos se basa en un modelo **Pub/Sub** (Publish/Subscribe) utilizando un **Event Broker** (ej. Upstash Kafka o Supabase Realtime). Esto permite desacoplar los servicios y procesar los eventos de forma 100 % asíncrona.

6.1 Flujo de Eventos

1. Un despacho cambia de estado en nuestro servicio (G6).
2. G6 publica un evento JSON (Publish) en el tópico global **shipment-events** del Broker.
3. Los servicios consumidores (G8, G7) que están suscritos (Subscribe) a este tópico reciben el mensaje asíncronamente.
4. El Broker se encarga de asegurar la entrega, eliminando el acoplamiento directo entre G6 y los consumidores.

7. Formato Estándar del Evento

Todos los eventos emitidos por G6 siguen esta estructura. Cada vez que un despacho cambie de estado, el servicio del Grupo 6 generará y publicará el siguiente payload JSON en el tópico global **shipment-events**. Los servicios consumidores (Grupo 8 y Grupo 7) deberán estar suscritos a dicho tópico para procesar la información de forma asíncrona:

Envelope del Evento (estructura común)

```
{
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440000",
  "eventType": "ShipmentDelivered",
  "version": "1.0",
  "occurredAt": "2026-06-11T15:00:00Z",
  "producer": "g6-despacho",
  "correlationId": "req-123e4567",
  "payload": {
    \"package_index\": 1,
    \"total_packages\": 2, ... }
}
```

8. Catálogo de Eventos

8.1 ShipmentCreated

Se emite cuando G5 crea exitosamente un nuevo despacho.

ShipmentCreated

```
{
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440001",
  "eventType": "ShipmentCreated",
  "version": "1.0",
```

Campo	Tipo	Descripción
eventId	string (UUID)	Identificador único del evento.
eventType	string	Tipo del evento (ver catálogo).
version	string	Versión del esquema del evento.
occurredAt	datetime	Momento del evento (ISO 8601).
producer	string	Siempre "g6-despacho".
correlationId	string	ID de correlación para trazar flujo.
payload	object	Datos específicos del evento.

Cuadro 6: Campos del envelope de evento.

```

"occurredAt": "2026-06-11T14:00:00Z",
"producer": "g6-despacho",
"correlationId": "req-123e4567",
"payload": {
  \"package_index\": 1,
  \"total_packages\": 2,
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2,
  "status": "PENDING",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
}

```

Consumidores: G8 (notifica al cliente), G7 (registro).

8.2 ShipmentInTransit

Se emite cuando el paquete sale a reparto.

ShipmentInTransit

```

{
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440002",
  "eventType": "ShipmentInTransit",
  "version": "1.0",
  "occurredAt": "2026-06-11T14:30:00Z",
  "producer": "g6-despacho",
  "correlationId": "req-123e4567",
  "payload": {
    \"package_index\": 1,
    \"total_packages\": 2,
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "PENDING",
    "new_status": "IN_TRANSIT",
    "customer_name": "María González",
    "city": "Santiago"
  }
}

```

```
}  
}
```

Consumidores: G8 (“Tu paquete va en camino”).

8.3 ShipmentDelivered

Se emite cuando el paquete es entregado exitosamente.



ShipmentDelivered

```
{  
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440003",  
  "eventType": "ShipmentDelivered",  
  "version": "1.0",  
  "occurredAt": "2026-06-11T15:00:00Z",  
  "producer": "g6-despacho",  
  "correlationId": "req-123e4567",  
  "payload": {  
    \"package_index\": 1,  
    \"total_packages\": 2,  
    "shipment_id": "SHP-a1b2c3d4",  
    "order_id": "ORD-20260611-001",  
    "previous_status": "IN_TRANSIT",  
    "new_status": "DELIVERED",  
    "customer_name": "María González",  
    "city": "Santiago",  
    "delivered_at": "2026-06-11T15:00:00Z"  
  }  
}
```

Consumidores: G8 (“Tu paquete ha llegado”), G7 (métricas).

8.4 ShipmentCancelled

Se emite cuando un despacho es cancelado antes de salir a reparto.



ShipmentCancelled

```
{  
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440004",  
  "eventType": "ShipmentCancelled",  
  "version": "1.0",  
  "occurredAt": "2026-06-11T14:10:00Z",  
  "producer": "g6-despacho",  
  "correlationId": "req-123e4567",  
  "payload": {  
    \"package_index\": 1,  
    \"total_packages\": 2,  
    "shipment_id": "SHP-a1b2c3d4",  
    "order_id": "ORD-20260611-001",  
    "previous_status": "PENDING",  
    "new_status": "CANCELLED",  
    "customer_name": "María González",  
    "reason": "Pedido anulado por el cliente"  
  }  
}
```

```
}  
}
```

Consumidores: G8 (notifica cancelación), G5 (confirma).

8.5 ShipmentFailed

Se emite cuando un intento de entrega falla.



ShipmentFailed

```
{  
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440005",  
  "eventType": "ShipmentFailed",  
  "version": "1.0",  
  "occurredAt": "2026-06-11T16:00:00Z",  
  "producer": "g6-despacho",  
  "correlationId": "req-123e4567",  
  "payload": {  
    \"package_index\": 1,  
    \"total_packages\": 2,  
    "shipment_id": "SHP-a1b2c3d4",  
    "order_id": "ORD-20260611-001",  
    "previous_status": "IN_TRANSIT",  
    "new_status": "FAILED",  
    "customer_name": "María González",  
    "reason": "Destinatario ausente"  
  }  
}
```

Consumidores: G8 (“Hubo un problema con tu entrega”), G5.

8.6 ShipmentReturned

Se emite cuando un paquete entregado es devuelto.



ShipmentReturned

```
{  
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440006",  
  "eventType": "ShipmentReturned",  
  "version": "1.0",  
  "occurredAt": "2026-06-12T10:00:00Z",  
  "producer": "g6-despacho",  
  "correlationId": "req-123e4567",  
  "payload": {  
    \"package_index\": 1,  
    \"total_packages\": 2,  
    "shipment_id": "SHP-a1b2c3d4",  
    "order_id": "ORD-20260611-001",  
    "previous_status": "DELIVERED",  
    "new_status": "RETURNED",  
    "customer_name": "María González",  
    "reason": "Producto incorrecto"  
  }  
}
```

```
}
```

Consumidores: G8, G5 (gestionar reembolso), G7 (métricas).

Parte III

Contratos Inter-Grupo

9. Contrato con G5 (Pedidos o Order Management) — Entrada

Este contrato define **qué datos nos envía G5** cuando un pedido es confirmado y necesita ser despachado.

9.1 Flujo de Integración

1. El cliente completa el checkout y el pago es procesado por G8.
2. G5 (Pedidos o Order Management) consolida el pedido y nos notifica para despacho.
3. G5 hace una solicitud HTTP POST a nuestro `/api/v1/shipments`.
4. Respondemos con el `shipment_id` y estado `PENDING`.
5. G5 almacena el `shipment_id` para tracking y consultas futuras.

9.2 JSON que G5 nos envía

Request de G5 hacia G6

```
{
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2
}
```

9.3 JSON que G6 le responde a G5

Response de G6 hacia G5

```
[
  \"SHP-1\",
  \"SHP-2\"
]
```

9.4 Preguntas pendientes para G5

Confirmar con G5

1. Formato de `order_id`: propuesto como string libre.
2. ¿Necesitan campos adicionales en la respuesta?
3. ¿Nos envían el peso o lo calculamos nosotros?
4. ¿Cancelan vía `PATCH` o necesitan endpoint dedicado?

10. Contrato con G8 (Pago Simulado y Notificaciones) — Salida

Define **qué datos le enviamos a G8** en cada cambio de estado.

10.1 Flujo de Integración

1. Un shipment cambia de estado (vía PATCH o simulación).
2. Construimos el evento JSON correspondiente.
3. Publicamos el evento en el tópico `shipment-events` del Event Broker.
4. G8 consume el evento desde el tópico y notifica al usuario final.

10.2 JSON que G6 publica en el Event Broker (para G8)

Evento publicado por G6 (ejemplo: entrega)

```
{
  "eventId": "evt-550e8400-e29b-41d4-a716-446655440003",
  "eventType": "ShipmentDelivered",
  "version": "1.0",
  "occurredAt": "2026-06-11T15:00:00Z",
  "producer": "g6-despacho",
  "correlationId": "req-123e4567",
  "payload": {
    \"package_index\": 1,
    \"total_packages\": 2,
    \"shipment_id\": \"SHP-a1b2c3d4\",
    \"order_id\": \"ORD-20260611-001\",
    \"previous_status\": \"IN_TRANSIT\",
    \"new_status\": \"DELIVERED\",
    \"customer_name\": \"María González\",
    \"city\": \"Santiago\",
    \"delivered_at\": \"2026-06-11T15:00:00Z\"
  }
}
```

10.3 Comportamiento esperado de G8 (Pago Simulado y Notificaciones)

- **Procesamiento:** El servicio del Grupo 8 consumirá el evento desde el tópico `shipment-events` de forma asíncrona. G6 no recibe ninguna respuesta directa ni código HTTP de parte de G8.
- **Acuse de recibo (ACK):** Una vez procesada la información localmente por su lógica interna, el componente consumidor de G8 emitirá un **ACK** (*Acknowledgment*) automático hacia el Event Broker para confirmar la recepción exitosa y permitir la liberación del mensaje en la cola. Este ACK es una señal de bajo nivel gestionada por la librería del Broker, no un payload JSON personalizado.
- **Acción final:** El Grupo 8 será responsable de gatillar la alerta o correo correspondiente al usuario final utilizando los datos del payload transmitido (`order_id`, `status`, `customer_name`).

Nota

En una arquitectura Pub/Sub real, el productor (G6) nunca ve el ACK del consumidor (G8). El ACK viaja exclusivamente entre el consumidor y el Broker, y su único efecto es marcar el mensaje como procesado en la cola. Esto garantiza el desacoplamiento total entre ambos servicios.

10.4 Preguntas pendientes para G8

! Confirmar con G8

1. ¿Campos adicionales necesarios? (email, teléfono)
2. ¿Qué eventos les interesan? ¿Los 6 o solo algunos?
3. Coordinar acceso y credenciales al Event Broker.

11. Contrato con G7 (Reportería, Batch y Streaming) — Salida

G7 consume datos para dashboards de rendimiento.

11.1 Opciones de Integración

1. **Vía API REST:** GET /shipments con filtros.
2. **Vía Eventos:** Suscripción al tópico shipment-events en el Broker.

11.2 Datos relevantes para G7

Métrica	Cómo la obtienen
Total despachos	GET /shipments → total.
Por estado	Filtrar con ?status=DELIVERED.
Tasa de éxito	DELIVERED / total.
Tasa de fallos	(FAILED + RETURNED) / total.
Tiempo de entrega	updated_at - created_at en DELIVERED.

12. Contrato con G1 (Frontend) — Consulta

G1 consume nuestro endpoint de tracking.

Endpoint: GET /api/v1/shipments/{shipment_id}

Campo	Uso en la UI
status	Paso actual en la barra de progreso.
estimated_delivery	“Llega el 14 de Junio”.
city	“Tu paquete va camino a Santiago”.
updated_at	“Última actualización: hace 30 min”.

Parte IV

Matriz de Dependencias y Arquitectura

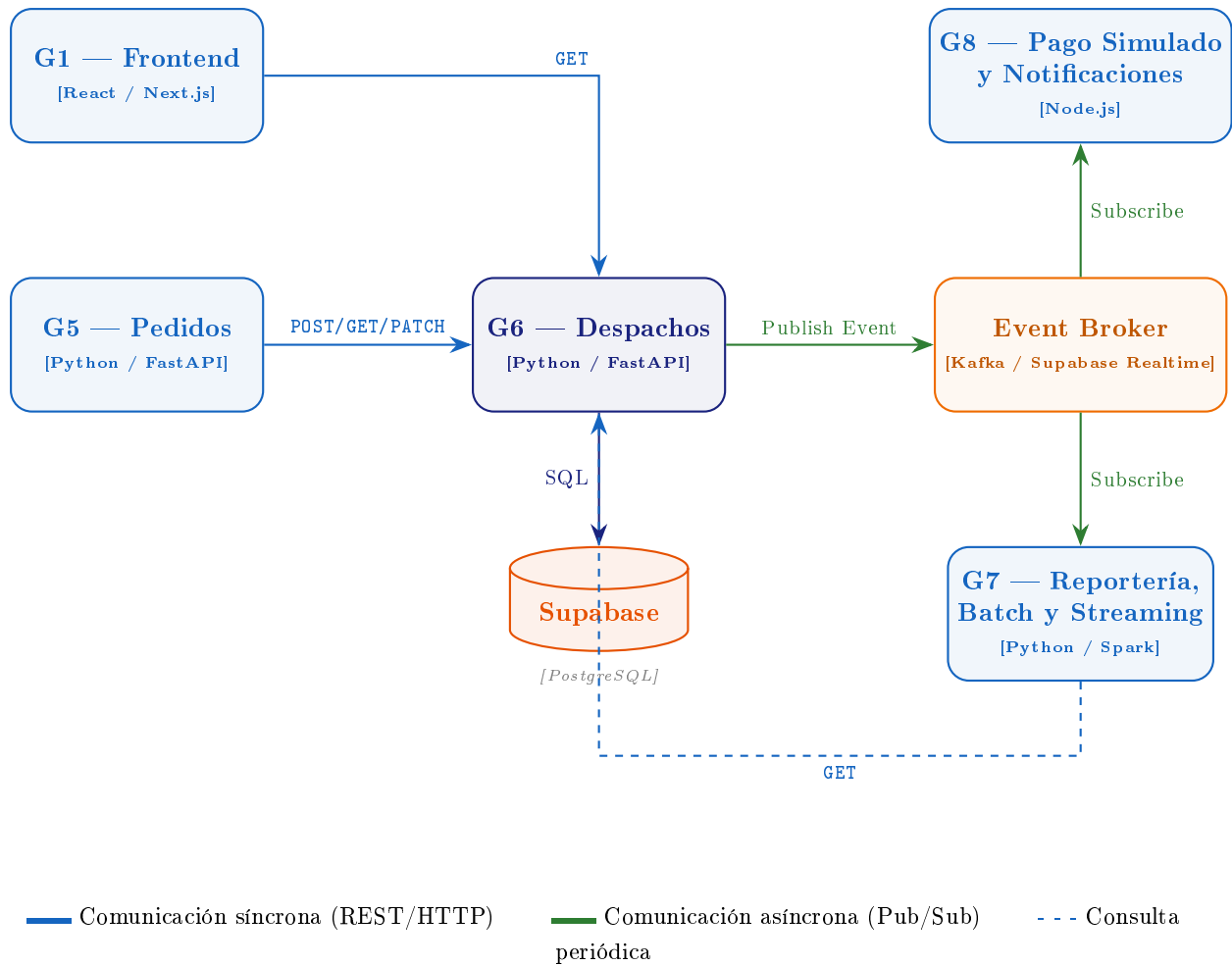
13. Matriz de Dependencias

Origen		Destino	Tipo	Protocolo	Datos
G5		G6	Síncrono	REST POST	Datos del pedido.
G5		G6	Síncrono	REST GET	Consulta por <code>order_id</code> .
G5		G6	Síncrono	REST PATCH	Cancelación.
G1		G6	Síncrono	REST GET	Tracking por <code>shipment_id</code> .
G6	Broker (Kafka/Supabase)		Asíncrono	Publish (Pub)	Mensaje en tópico <code>shipment-events</code>
Broker	G8 (Pago Simulado y Notificaciones)		Asíncrono	Subscribe (Sub)	Consume desde tópico <code>shipment-events</code>
Broker	G7 (Reportería, Batch y Streaming)		Asíncrono	Subscribe (Sub)	Consume desde tópico <code>shipment-events</code>
G7		G6	Síncrono	REST GET	Lista para reportes.

Cuadro 7: Matriz completa de dependencias del Grupo 6.

14. Diagrama de Arquitectura (C4 — Nivel 2: Contenedores)

System Container Diagram — Ecosistema Logístico (perspectiva G6 Despachos)



15. Esquema SQL de la Base de Datos

```

CREATE TABLE shipments (
  shipment_id TEXT PRIMARY KEY,
  order_id TEXT NOT NULL,
  customer_name TEXT NOT NULL,
  address TEXT NOT NULL,
  city TEXT NOT NULL,
  origin_cd TEXT NOT NULL,
  volumetric_weight REAL NOT NULL,
  shipping_cost INTEGER NOT NULL,
  weight_kg REAL NOT NULL CHECK (weight_kg > 0),
  status TEXT NOT NULL DEFAULT 'PENDING'
  CHECK (status IN (
    'PENDING', 'IN_TRANSIT', 'DELIVERED',
    'CANCELLED', 'FAILED', 'RETURNED'
  )),
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
  estimated_delivery TIMESTAMP
);

```

i Nota

Esquema para Supabase (PostgreSQL). Si se usa Firebase, se adaptará a documentos JSON con los mismos campos.

16. Supuestos, Riesgos y Mitigaciones

16.1 Supuestos

1. **Formato de entrada estandarizado:** G5 (Pedidos) envía los datos del pedido con todos los campos obligatorios definidos en el contrato REST (`order_id`, `customer_name`, `address`, `city`, `weight_kg`).
2. **Idempotencia del `order_id`:** Cada pedido genera exactamente un envío. Si G5 reintenta el POST, el sistema rechaza el duplicado con `409 Conflict`.
3. **Disponibilidad del Event Broker:** El Event Broker (Kafka o Supabase Realtime) está operativo y accesible. La publicación de eventos es *at-least-once*.
4. **Base de datos centralizada:** G6 es el único grupo que escribe en la tabla `shipments`. Otros grupos solo consultan vía REST.
5. **Conectividad cloud:** Todos los servicios se despliegan en la nube y se comunican a través de HTTPS o protocolo de mensajería seguro.
6. **Autenticación delegada:** La autenticación del usuario final la gestiona G1/G8. G6 confía en los headers `X-Consumer` y `X-Correlation-Id` para trazabilidad.

16.2 Riesgos y Mitigaciones

ID	Riesgo	Mitigación	Impacto
R1	Caída del Event Broker impide que G8 notifique al usuario final.	Cola de reintentos con backoff exponencial; tabla <code>outbox</code> local para eventos pendientes.	Alto
R2	G5 envía datos incompletos o con formato inválido.	Validación estricta en el endpoint <code>POST /shipments</code> con retorno <code>422</code> y mensaje descriptivo.	Medio
R3	Duplicación de envíos por reintentos de G5 sin <code>Idempotency-Key</code> .	Constraint <code>UNIQUE</code> en <code>order_id</code> a nivel de base de datos y verificación del header.	Medio
R4	Latencia alta en la base de datos Supabase por volumen de consultas de G7.	Índices en <code>status</code> y <code>created_at</code> ; endpoint paginado con <code>limit/offset</code> .	Medio
R5	Transición de estado inválida (e.g., de <code>DELIVERED</code> a <code>PENDING</code>).	Máquina de estados con validación explícita de transiciones permitidas antes de aplicar el <code>PATCH</code> .	Bajo
R6	Pérdida de trazabilidad entre servicios ante fallos parciales.	Propagación obligatoria de <code>X-Correlation-Id</code> en todos los requests y eventos publicados.	Medio

Cuadro 8: Riesgos identificados para el servicio de Despachos (G6).

16.3 Manejo de Errores

Todos los errores retornados por la API siguen la estructura estándar definida en la Sección 2 (Información General). Además:

- Los errores de validación (422) incluyen el campo específico que falló en el **message**.
- Los conflictos de idempotencia (409) retornan el **shipment_id** existente para que el consumidor pueda recuperarse.
- Los errores de transición de estado inválida (422) indican el estado actual y los estados de destino válidos.
- Todos los errores incluyen el **correlationId** para facilitar la depuración entre grupos.

17. Historial de Versiones

Ver.	Fecha	Autor	Cambios
1.0	16 Jun 2026	Grupo 6	Versión inicial.
1.1	17 Jun 2026	Grupo 6	Soporte Multi-Origen y Tarifas por Peso Volumétrico (v1.1).